

Reproduction Guide

Origin-Destination Trajectory Prediction with Deep Learning and Markov Models

This document provides step-by-step instructions for reproducing the data processing workflow, route prediction results, evaluation metrics, statistical tests, and figures reported in the manuscript. All file paths are relative paths.

1. Software Environment

The codes were executed in a Linux-based Conda environment.

Main environment

- Operating system: Linux
- Python version: 3.10.13

Main Python packages

numpy==1.23.5, pandas==2.3.2, geopandas==0.14.4, networkx==3.4, shapely==2.0.1,
matplotlib==3.9.1, seaborn==0.13.2, scipy==1.15.2, statsmodels==0.14.5, osmnx==1.9.3,
contextily==1.6.2, torch==2.4.1, torch-geometric==2.6.1, torch-scatter==2.1.2,
torch-sparse==0.6.18, torch-cluster==1.6.3, tqdm==4.67.1

2. Repository Structure

This repository is divided into two main folders: For_test and Figure_and_Real.

The For_test folder contains the codes and sample data required to reproduce the overall workflow of the study. Because the full original dataset is too large to upload, the data included in For_test correspond to 1/10 of the original dataset. The purpose of this folder is to allow reviewers to execute the complete analytical procedure, including data preprocessing, route prediction, and accuracy calculation.

The Figure_and_Real folder contains the codes used to reproduce the figures and statistical results reported in the manuscript. The input files used in this folder are based on the final outputs generated from the full original dataset used in the actual study. Therefore, while For_test demonstrates the full reproducible workflow using the reduced dataset, Figure_and_Real provides the final result files needed to reproduce the figures and statistics reported in the manuscript.

3. Data Index

The following data IDs are used to indicate when the same dataset is reused by multiple scripts.

Data ID	Dataset	Description	Path
Data 1	Road network graph	Filtered Jeju driving road network graph	road_network/jeju_drive_filtered_int_code.graphml
Data 2	Base hexagon grid	Hexagon grid used for constructing the hexagon	new_hexagraph/jeju_hexa_for_use.shp

Data ID	Dataset	Description	Path
		road network	
Data 3	Hexagon road network	Hexagon-based road network generated from Data 1 and Data 2	new_hexagraph/hexa_network_with_road
Data 4	Jeju boundary	Boundary shapefile within Jeju	jeju_shp/jeju.shp
Data 5	Raw GPS data	Original GPS trajectory CSV files	Jeju_data/Training/gps/ Jeju_data/Validation/gps/
Data 6	Jeju-filtered GPS data	GPS files after filtering points located in Jeju	Jeju_data/Training/gps_split_jeju/ Jeju_data/Validation/gps_split_jeju/
Data 7	Hexagon-converted GPS data	GPS trajectories converted into hexagon cell sequences	Jeju_data/Training/gps_hexa/ Jeju_data/Validation/gps_hexa/
Data 8	Original traveler-related tables	Original survey and trip-related input CSV files	Jeju_data/Training/tn_*.csv Jeju_data/Validation/tn_*.csv (traveler survey and trip-related tables)
Data 9	Traveler property table	Traveler-level feature table	Jeju_data/Training/properties_for_traveler.csv Jeju_data/Validation/properties_for_traveler.csv
Data 10	Connected GPS shortcut routes	Topologically corrected GPS route sequences	Jeju_data/Training/shortcut_route/gps_shortcut.csv Jeju_data/Validation/shortcut_route/gps_shortcut.csv
Data 11	OD route split files	OD segment files for K=8, 12, and 16	Jeju_data/Training/shortcut_route/route_split_{k}/ Jeju_data/Validation/shortcut_route/route_split_{k}/
Data 12	Traveler OD feature files	Traveler features matched with OD segment information	Jeju_data/Training/shortcut_route/traveler_proper_OD_{k}.csv Jeju_data/Validation/shortcut_route/traveler_proper_OD_{k}.csv
Data 13	Shortest path predictions	Prediction output of the shortest path model	shortcut_prediction_route/shortcut_path_{k}.csv

Data ID	Dataset	Description	Path
Data 14	Basic Markov predictions	Prediction output of the Basic Markov model	prediction_markov/markov_route_{k}.csv prediction_markov/markov_route_split_{k}.csv
Data 15	Conditional Markov predictions	Prediction output of the attribute-based Markov model	prediction_markov_2/markov2_route_{k}.csv prediction_markov_2/markov2_route_split_{k}.csv
Data 16	LSTM model weights	Example LSTM model weights	weight_for_test/weight_{k}/model_epoch_{n}.pth
Data 17	LSTM model weights used in actual study	Final LSTM model weights used in the actual study	weight/model_{k}.pth
Data 18	LSTM predictions	Raw LSTM prediction output	prediction_LSTM/LSTM_{k}_by1.csv
Data 19	Refined LSTM predictions	LSTM prediction after topological refinement	prediction_LSTM/LSTM_fix_{k}_by1.csv
Data 20	Merged validation routes	Ground-truth validation routes merged by traveler	merged_routes_by_{k}.csv Figure_and_Real/trajectory/merged_routes_by_{k}.csv
Data 21	Accuracy results	Accuracy results for K=8, 12, and 16	accuracy/accuracy_ver_last_{k}.csv
Data 22	Figure 1 input data	Ground-truth trajectory data used for Figure 1	Figure_and_Real/GPS_trajectory_for_figure/Figure_1/merged_routes_by_{k}.csv
Data 23	Figure 6 input data	Route comparison data for Figure 6	Figure_and_Real/GPS_trajectory_for_figure/Figure_6/
Data 24	Figure 9 input data	Route comparison groups for Figure 9	Figure_and_Real/GPS_trajectory_for_figure/Figure_9/group_1/ Figure_and_Real/GPS_trajectory_for_figure/Figure_9/group_2/
Data 25	Figure 10 input data	Trajectory data for Figure 10	Figure_and_Real/GPS_trajectory_for_figure/Figure_10/
Data 26	Figure 11 input data	Trajectory data for Figure 11	Figure_and_Real/GPS_trajectory_for_figure/Figure_11/

4. Code and Data

Code file	Purpose	Input data	Output data
For_test/code1_hexa_network.ipynb	Show the hexagon road network	Data 1, 2, 3	-
For_test/code2_change_gps_to_hexa.ipynb	Filter GPS points within Jeju and convert GPS trajectories to hexagon sequences	Data 3, 4, 5	Data 6, 7
For_test/code3_create_traveler_feature.ipynb	Create traveler-level property tables	Data 7, 8	Data 9
For_test/code4_create_od_route_segments.ipynb	Generate topologically connected routes and split them into OD route segments	Data 3, 7, 9	Data 10, 11, 12
For_test/code5_predict_shortest.ipynb	Predict validation OD routes using shortest path algorithm	Data 3, 11	Data 13
For_test/code6_predict_markov.ipynb	Predict validation OD routes using the Basic Markov model	Data 3, 10, 11	Data 14
For_test/code6.5_predict_attribute_markov.ipynb	Predict validation OD routes using the Conditional Markov model	Data 3, 9, 10, 11	Data 15
For_test/code7_train_LSTM_example.ipynb	Train the LSTM model example for K=8	Data 3, 11, 12	Data 16
For_test/code8_predict_LSTM.py	Predict validation OD routes using trained Hybrid GCN-LSTM models	Data 3, 11, 12, 17	Data 18
For_test/code8_predict_LSTM.sh	SLURM script for executing code8_predict_LSTM.py	-	-
For_test/code9_refine_LSTM.ipynb	Refine LSTM predictions using graph topology	Data 3, Data 18	Data 19
For_test/code10_accuracy.ipynb	Compute Jaccard Similarity and Levenshtein Distance	Data 11, 13, 14, 15, 19, 20	Data 20, 21
Figure_and_Real/Evaluation_and_significance.ipynb	Generate performance plots and statistical significance results	Route prediction files under Figure_and_Real/trajectory/	Figures 7–8 and statistical test outputs
Figure_and_Real/drawing_picture.ipynb	Generate route visualization figures	Data 2, Data 3, Data 22–26	Figures 1, 6, 9, 10, and 11

5. Overall Workflow

The full workflow consists of the following stages:

1. Construct and show the hexagon road network. (For_test/code1)
2. Filter raw GPS points located within Jeju. (For_test/code2)
3. Convert GPS trajectories to hexagon cell sequences. (For_test/code2)
4. Create traveler-level property tables. (For_test/code3)
5. Generate connected shortcut routes and split them into OD segments. (For_test/code4)
6. Predict routes using four models:
 - Shortest Path (For_test/code5)
 - Basic Markov (For_test/code6)
 - Conditional Markov (For_test/code6.5)
 - LSTM (For_test/code7, 8)
7. Refine LSTM predictions using graph topology. (For_test/code9)
8. Compute Jaccard Similarity and Levenshtein Distance. (For_test/code10, Figure_and_Real/Evaluation_and_significance.ipynb)
9. Conduct statistical significance tests. (Figure_and_Real/Evaluation_and_significance.ipynb)
10. Generate manuscript figures. (Figure_and_Real/drawing_picture.ipynb)

6. Recommended Execution Order

Run the files in the following order:

1. For_test/code1_hexa_network.ipynb
2. For_test/code2_change_gps_to_hexa.ipynb
3. For_test/code3_create_traveler_feature.ipynb
4. For_test/code4_create_od_route_segments.ipynb
5. For_test/code5_predict_shortest.ipynb
6. For_test/code6_predict_markov.ipynb
7. For_test/code6.5_predict_attribute_markov.ipynb
8. For_test/code7_train_LSTM_example.ipynb
9. For_test/code8_predict_LSTM.py
10. For_test/code9_refine_LSTM.ipynb
11. For_test/code10_accuracy.ipynb
12. Figure_and_Real/Evaluation_and_significance.ipynb
13. Figure_and_Real/drawing_picture.ipynb

7. Description of Each Code File

For further details regarding the implementation and workflow, please refer to the comments provided at the beginning of each code file in the repository.

7.1 For_test/code1_hexa_network.ipynb

Input data

The Jeju road network graph and the base hexagon grid.

Output data

The hexagon road network used throughout the study.

Description

This notebook shows the hexagon-based road network used in the study. The original road network is integrated with the hexagon grid to create a graph structure that preserves road connectivity while representing trajectories as hexagon sequences. The resulting network serves as the spatial framework for trajectory conversion, route prediction, route refinement, and visualization.

7.2 For_test/code2_change_gps_to_hexa.ipynb

Input data

Raw GPS trajectories, the Jeju boundary, and the hexagon road network.

Output data

Jeju-filtered GPS trajectories and hexagon-converted trajectory sequences.

Description

This notebook preprocesses the original GPS trajectory data. GPS points located outside Jeju are removed, and the remaining trajectories are converted into sequences of hexagon cells using the hexagon road network. This conversion produces the network-based trajectory representation used in all subsequent analyses.

7.3 For_test/code3_create_traveler_feature.ipynb

Input data

Hexagon-converted GPS trajectories and traveler-related survey tables.

Output data

Traveler property tables.

Description

This notebook generates traveler-level feature tables by combining processed trajectory data with traveler survey information. The resulting attributes are used to characterize travelers and are later incorporated into the Conditional Markov and LSTM-based prediction models.

7.4 For_test/code4_create_od_route_segments.ipynb

Input data

Hexagon-converted GPS trajectories, traveler property tables, and the hexagon road network.

Output data

Connected route sequences, OD route segment files, and traveler-level OD feature tables.

Description

This notebook generates topologically connected routes and splits them into OD route segments. Because raw trajectory sequences may contain disconnected transitions, route connectivity is first corrected using the hexagon road network. The resulting routes are then divided into OD segments according to the specified split size ($k=8$, $k=12$, and $k=16$). These OD segment files serve as the primary inputs for all route prediction models used in the study.

7.5 For_test/code5_predict_shortest.ipynb

Input data

The hexagon road network and validation OD route segments.

Output data

Shortest path route prediction files.

Description

This notebook generates route predictions using the shortest path algorithm. For each OD segment, the shortest path between the origin and destination is calculated on the hexagon road network. The resulting routes serve as a baseline model for comparison with the data-driven prediction approaches (Markov, Conditional Markov, and LSTM).

7.6 For_test/code6_predict_markov.ipynb

Input data

Training route sequences, validation OD route segments, and the hexagon road network.

Output data

Basic Markov route prediction files.

Description

This notebook implements the Basic Markov model. Transition probabilities are estimated from the training route sequences, and validation routes are predicted using these learned transition patterns. The model captures common movement behaviors observed in the training trajectories.

7.7 For_test/code6.5_predict_attribute_markov.ipynb

Input data

Traveler property tables, training route sequences, validation OD route segments, and the hexagon road network.

Output data

Conditional Markov route prediction files.

Description

This notebook implements the Conditional Markov model. In contrast to the Basic Markov model, traveler attributes are incorporated into the transition estimation process. This allows route predictions to reflect both observed movement patterns and traveler characteristics.

7.8 For_test/code7_train_LSTM_example.ipynb

Input data

The hexagon road network, OD route segment files, and traveler-level OD feature tables.

Output data

Example trained LSTM model weights. The model weights generated by 'code7_train_LSTM_example.ipynb' are provided only as an example.

Description

This notebook demonstrates the training procedure of the LSTM-based route prediction model using the reduced dataset provided in the repository. The model learns route patterns from OD segments, traveler information, and graph-based spatial context. The notebook is included to illustrate the complete model training workflow.

In the shared repository, this notebook is provided as an example implementation in '.ipynb' format and demonstrates the training procedure only for the k=8 setting. In the actual study, the models for k=8, 12, and 16 were trained separately using '.py' and '.sh' scripts in a SLURM-based computing environment. Therefore, this notebook is intended to explain the training workflow step by step, while the trained model weights used in the study are provided separately.

7.9 For_test/code8_predict_LSTM.py

Input data

The hexagon road network, validation OD route segments, traveler-level OD feature tables, and trained LSTM model weights. The prediction scripts use the trained model weights from the actual study stored in './weight/', rather than the example weights stored in './weight_for_test/'.

Output data

LSTM route prediction files.

Description

This script generates route predictions using the trained LSTM model. The model predicts routes for each OD segment while considering traveler attributes and graph structure information. Adjacency-constrained beam search is used during decoding to ensure that predicted routes follow the connectivity of the hexagon road network. Segment-level predictions are subsequently merged into complete route sequences.

7.10 For_test/code8_predict_LSTM.sh

Input data

The same inputs required by the LSTM prediction script.

Output data

LSTM route prediction files and execution logs.

Description

This shell script is provided for running the LSTM prediction process on a Linux server using SLURM. It specifies the computing resources required for execution and calls the Python prediction script automatically.

In the actual study, both the training and validation procedures corresponding to the example workflow shown in ‘code7_train_LSTM_example.ipynb’ were conducted using similar ‘.py’ and ‘.sh’ files in a SLURM-based computing environment.

7.11 For_test/code9_refine_LSTM.ipynb

Input data

The hexagon road network and raw LSTM prediction files.

Output data

Refined LSTM route prediction files.

Description

This notebook refines the raw LSTM prediction results using the topology of the hexagon road network. Any disconnected transitions that remain after prediction are corrected to ensure that all predicted routes form valid connected paths on the network.

7.12 For_test/code10_accuracy.ipynb

Input data

Ground-truth validation routes and route predictions generated by the Shortest Path, Basic Markov, Conditional Markov, and LSTM models.

Output data

Accuracy result files and evaluation statistics.

Description

This notebook evaluates route prediction performance by comparing predicted routes with observed validation trajectories. Jaccard Similarity and Levenshtein Distance are calculated for each model and each split size. The resulting accuracy files are later used to reproduce the performance tables, statistical tests, and evaluation figures reported in the manuscript.

7.13 Figure_and_Real/Evaluation_and_significance.ipynb

Input data

Route prediction results and accuracy files generated from the full original dataset used in the actual study, rather than the reduced dataset provided in the For_test folder.

Output data

Performance figures and statistical significance test results.

Description

This notebook reproduces the quantitative evaluation results reported in the manuscript. Unlike the For_test workflow, which uses a reduced dataset for demonstration purposes, this notebook uses the final route prediction results generated from the full original dataset used in the actual study. It generates performance comparison figures and performs Friedman tests, pairwise Wilcoxon signed-rank tests, and Holm correction procedures to assess whether differences among prediction models are statistically significant.

7.14 Figure_and_Real/drawing_picture.ipynb

Input data

The hexagon road network and figure-specific trajectory datasets generated from the full dataset.

Output data

Figures reported in the manuscript.

Description

This notebook reproduces the route visualization figures presented in the manuscript. Although each figure uses a separate dataset prepared specifically for visualization, all of these datasets were generated from the full original dataset used in the actual study. The generation procedures are identical to those demonstrated in the For_test workflow, but were applied to the complete dataset rather than the reduced sample dataset.

- For Figure 1, the visualization is based on the ground-truth route dataset ('merged_routes_by_12.csv') generated during the accuracy evaluation procedure in 'For_test/code10_accuracy.ipynb'.
- For Figure 6, the visualization uses route predictions generated by the Shortest Path, Basic Markov, Conditional Markov, and LSTM models ('For_test/code5~code9'), together with the ground-truth route dataset generated in 'For_test/code10_accuracy.ipynb'. A representative traveler was selected and visualized for comparison.
- For Figure 9, the visualization is based on OD route segments generated in 'For_test/code4_create_od_route_segments.ipynb'.
- For Figures 10 and 11, the visualizations use route predictions generated by the four prediction models ('For_test/code5~code9') together with route segments generated in 'For_test/code4_create_od_route_segments.ipynb'. Representative examples were selected and visualized for comparison.

Therefore, while the datasets stored in the 'Figure_and_Real' folder are provided specifically for reproducing the manuscript figures, all of them originate from the same workflow described in the 'For_test' folder and were generated using the full original dataset employed in the study.